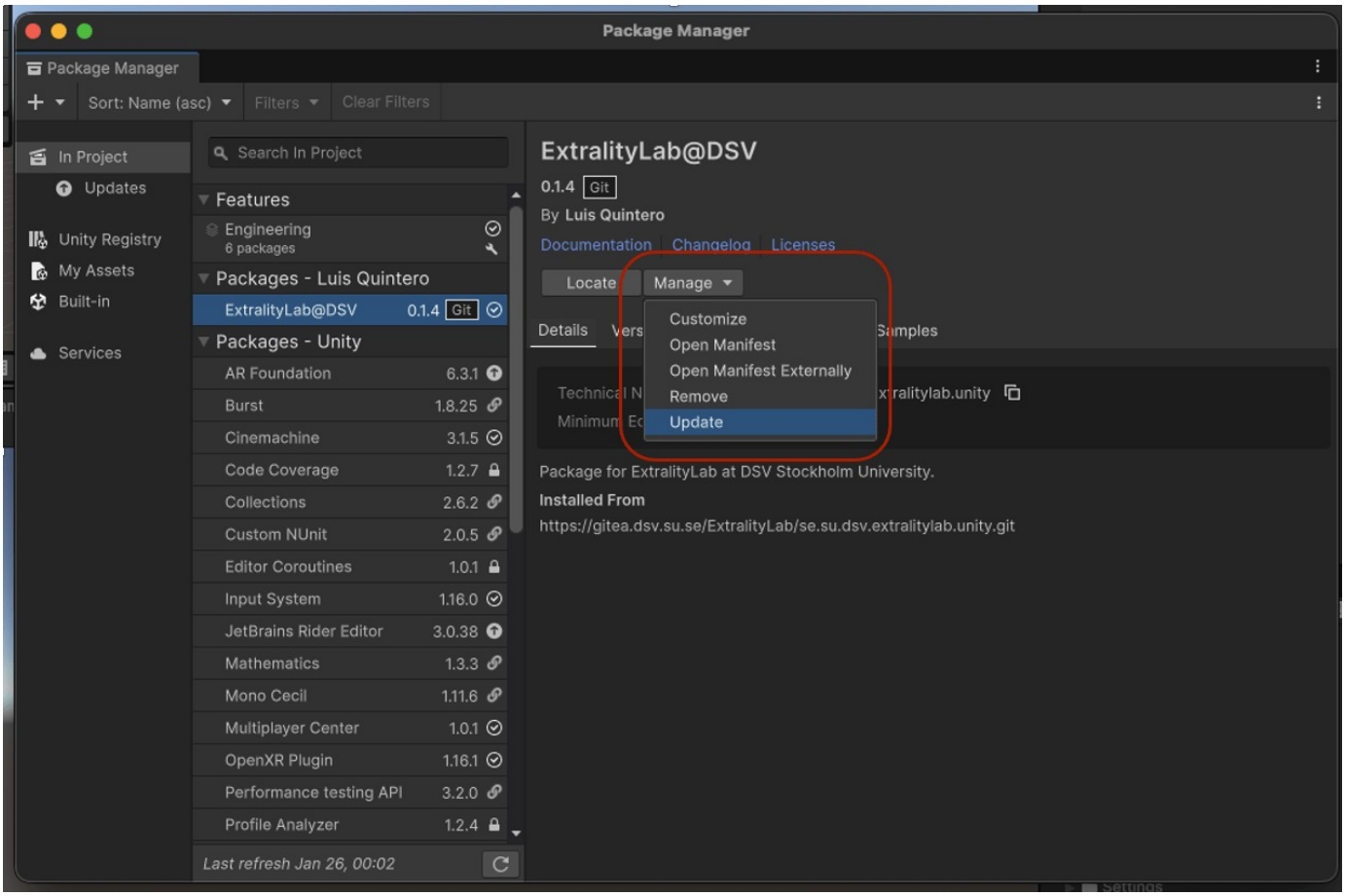
Lab 5: Instructions

In this lab, we will merge immersive experiences and tangible interaction.

Before starting this lab, ensure you have completed all the required steps from the Required Preparation section

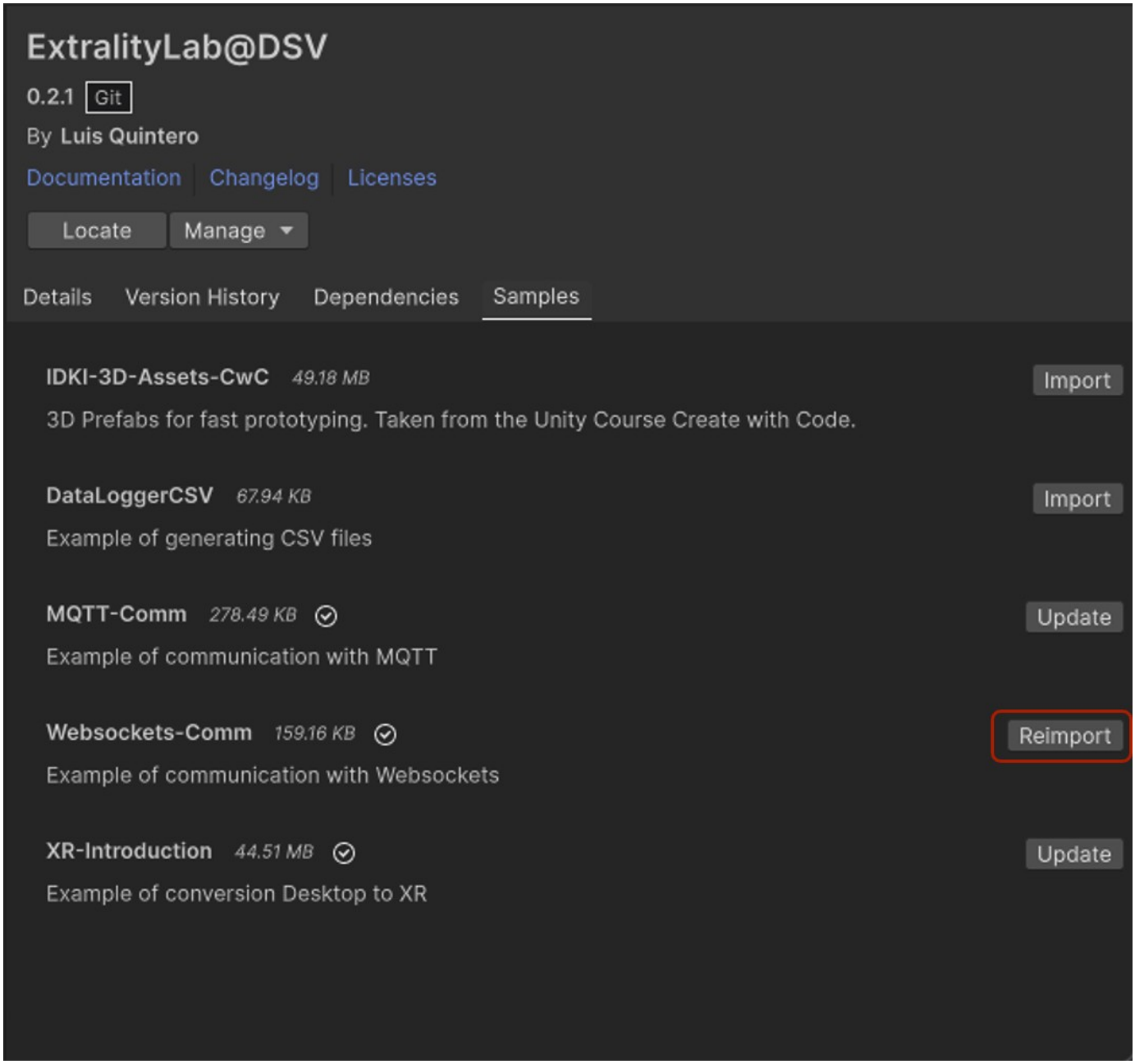
Continue on the same Unity project used for Lab 4.

- Open the same Unity project used in Lab 4
- Make sure that you `git commit` before starting, and remember to git commit continuously in case something goes wrong while developing.
- We have been working with the Extrality Lab package in Unity (**v0.1.6**) from:
  https://gitea.dsv.su.se/ExtralityLab/se.su.dsv.extralitylab.unity.git
- If you are not working on the recommended version, follow:
  - Go to "Package Manager" and then select "In Project"
  - Find the "ExtralityLab@DSV" package and click "Manage" followed by "Update"
  - Your package should have been successfully updated.



Task 1: Import the Websockets-Comm Sample and open it

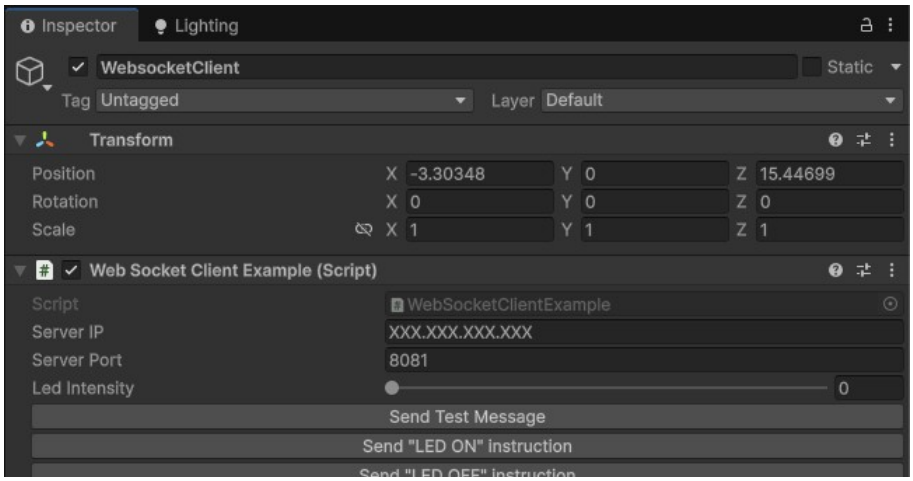
- Under "Samples", find the Websockets sample and import it to your project

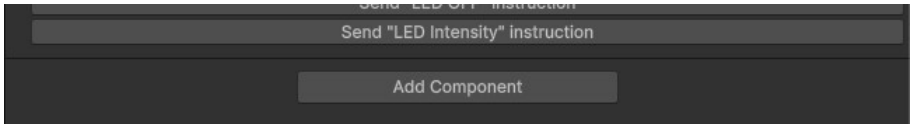


- Once imported, find the "SimpleWebSocketSample" scene and open it (This can be found under the samples folder e.g. **"Samples/ ExtralityLab@DSV/0.2.1/Websockets-Comm/03_Client_Example_Basic"**)

Task 2: Understanding the sample Scene

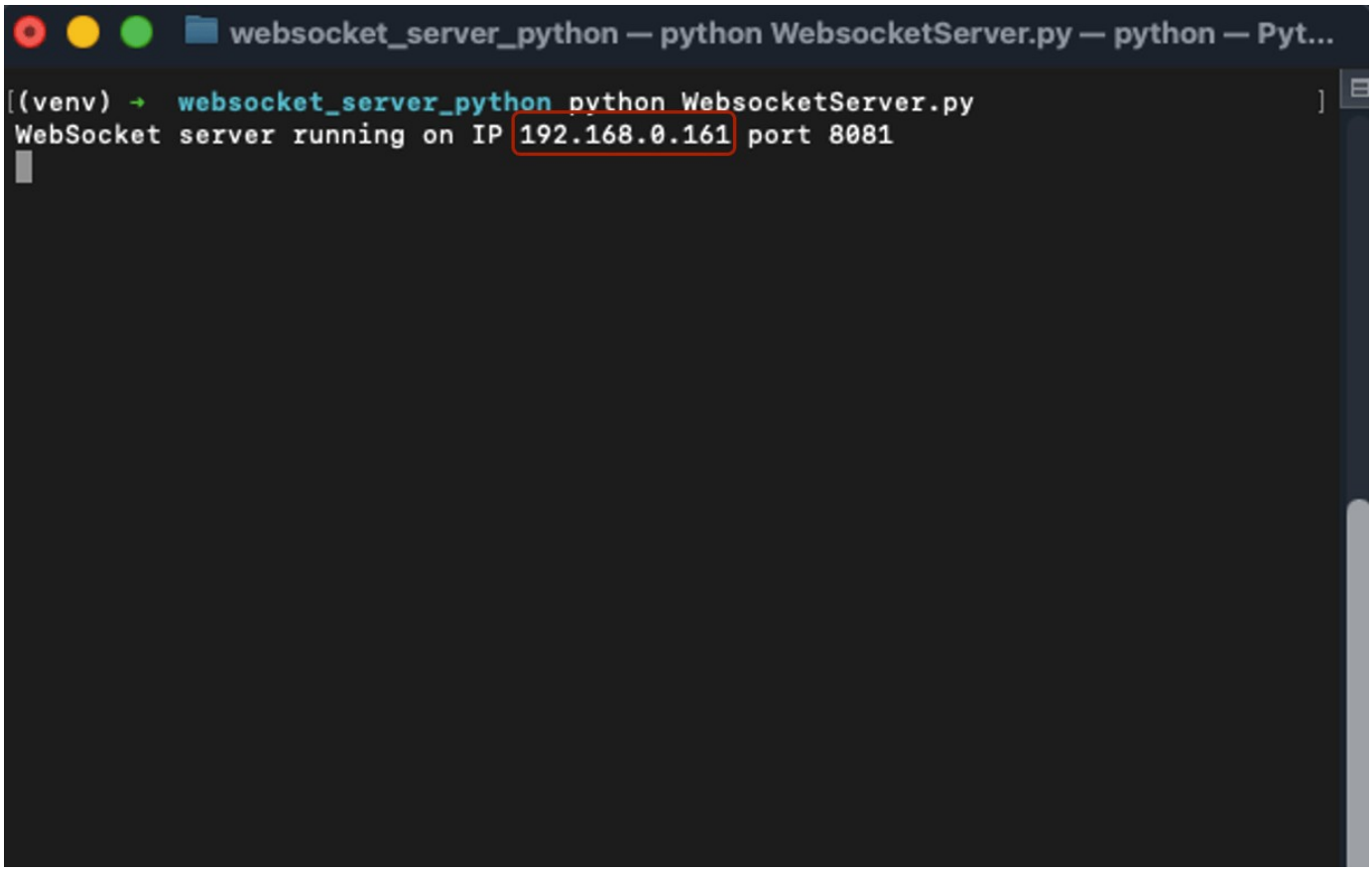
- This Websocket sample is very simple, containing only the default Main Camera and Directional Light GameObjects, and our **WebsocketClient Gameobject**
- Clicking on it reveals that it has a **"WebSocketClientExample"** script. This script contains the basics for communicating with a Websocket Server and our ESP32





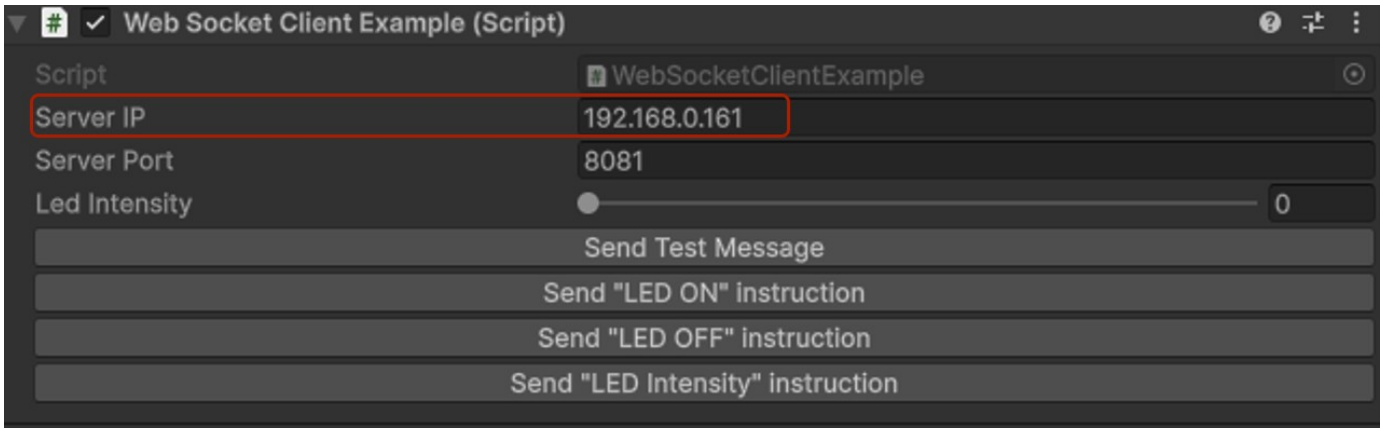
Task 3: Running and testing the Websocket Server and connecting Unity to it

- Assuming you have followed the Pre-requisite section for this lab, navigate to the folder inside the **"Tangible Interaction"** package and find the Python script responsible for starting our server (e.g. **"tangible-interaction/Websockets-Communication/01_Server_Python/WebsocketServerExample.py"**
  - **Note:** This folder is **NOT** on the Unity Project
- Start the server by opening a terminal window and running the correct Python command (e.g. **"python WebsocketServer.py"**)
- You should see the following on your terminal window: (**Important!** This window must be kept open during the entire duration of the lab)



Note!: This IP number will be different for everyone!

- Copy the IP number you see on your terminal, and paste it into the Unity Websocket Client script
- Run the Unity game and press the **"Send Test Message"** button



- Your terminal window should now contain two new messages, one informing you that a new client has connected, and another containing a message from Unity!

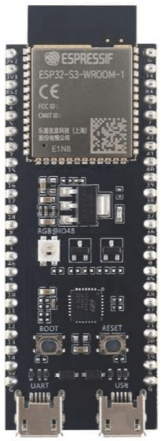
```
WebSocket server running on IP 192.168.0.161 port 8081
Client connected
Received: Device (Unity):C598 ... Device's Unique Identifier: FE8E292F-D4CE-579C
-ADC5-1E186F79204D
Received: Hello from Unity
```

Task 4: Uploading the ESP32 Websocket Client code and connecting it to the Websocket Server

- Grab one of the ESP32's we have available at the lab. We have 4 different types, but they are fundamentally all the same for this project. Make sure you grab one of the following, as well as the correct cable to connect it to your computer (USB-C or USB Micro)

Esp32

Esp32 S2



Esp32 S3

Esp32 S3 Zero

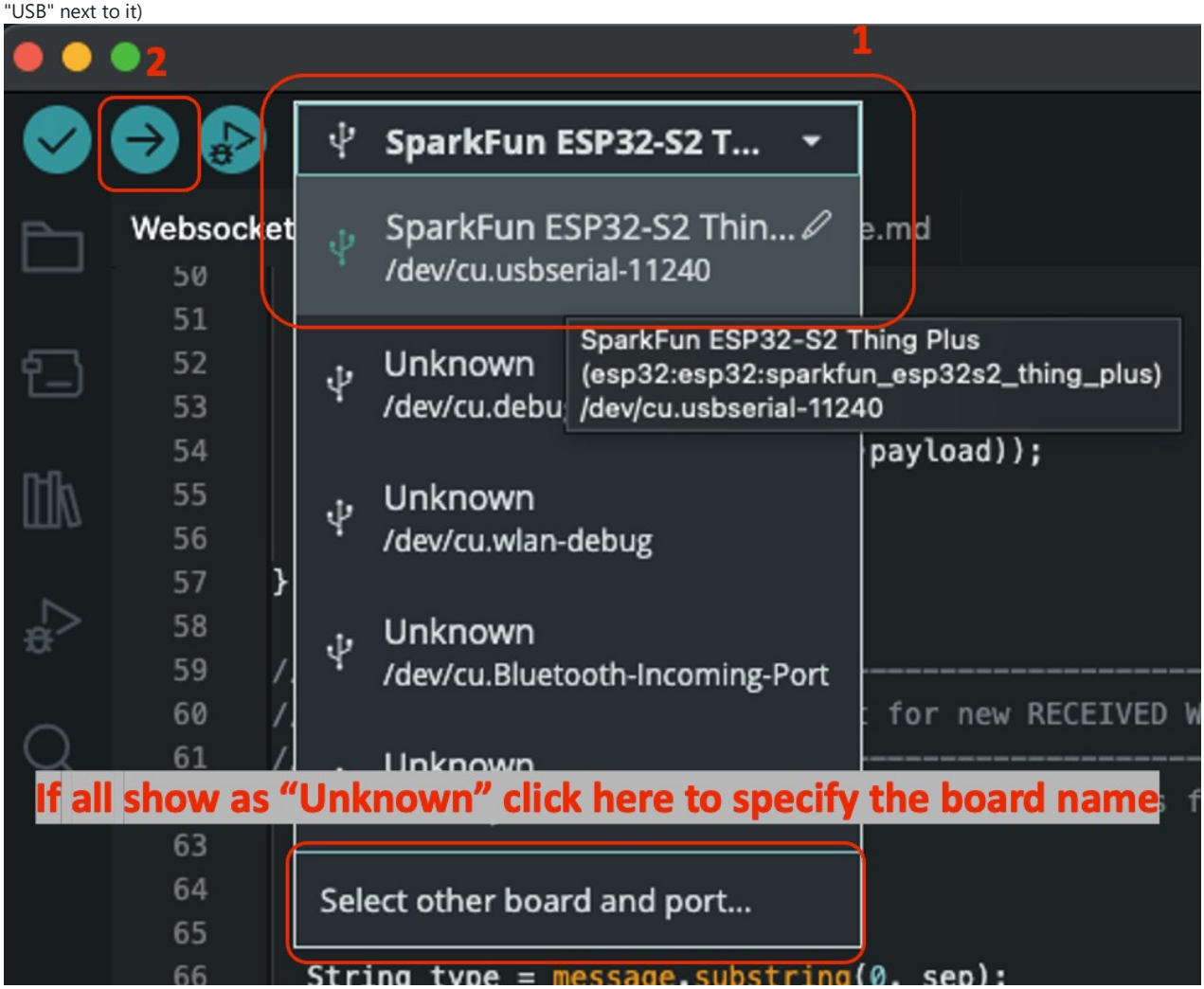
⚠️ **Note:** It's possible that due to the number of students and the limited number of available devices, you might have to group in pairs or groups of 3's for the tasks from here on out

- In the Tangible Interaction sample downloaded from Gitea, navigate to the **"Websockets-Communication/03_Client_Example_Basic"** folder
- Open the **"03_Client_Example_Basic.ino"** in the Arduino IDE
- Modify lines 21-24 to contain the following:
  - ssid: Wifi name
  - password: Wifi password
  - serverIP: The IP from the Python server
  - serverPort: Leave as 8081 unless you changed the port number in the Python script

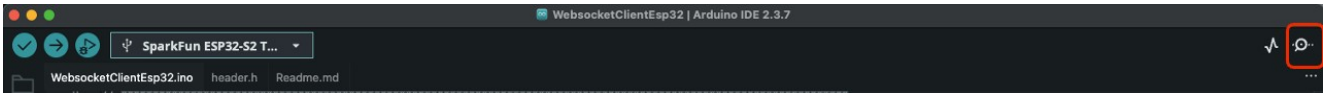
```
#include <WiFi.h>
#include <WebSocketsClient.h>
#include <Adafruit_NeoPixel.h>
#include <header.h>

// ------------------------------------------------------------
// WiFi / Server Settings                    Replace these
// ------------------------------------------------------------
const char* ssid = "WIFI_NAME_HERE"; //Replace with your Wifi's name
const char* password = "WIFI_PASSWORD_HERE"; //Replace with your Wifi's password
const char* serverIP = "SERVER_IP_HERE"; //Replace with your Python server's IP (e.g. 192.168.1.111)
```

```
const uint16_t serverPort = 8081; //Replace with your desired Port (or keep as is)
```

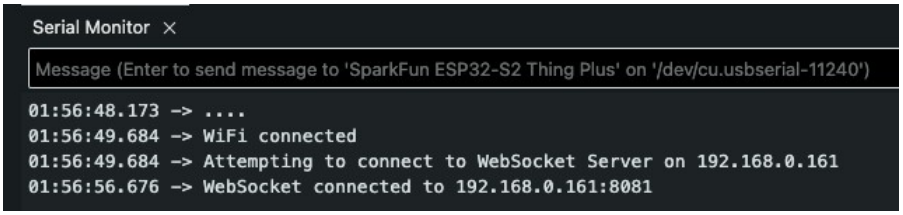
- Choose your board as the upload target and then click on the "Upload" button
  - **Important!:** If you are using the ESP32 S3 (Black one with 2 Micro USB Ports), make sure you connect your cable to the port that says "USB" next to it)

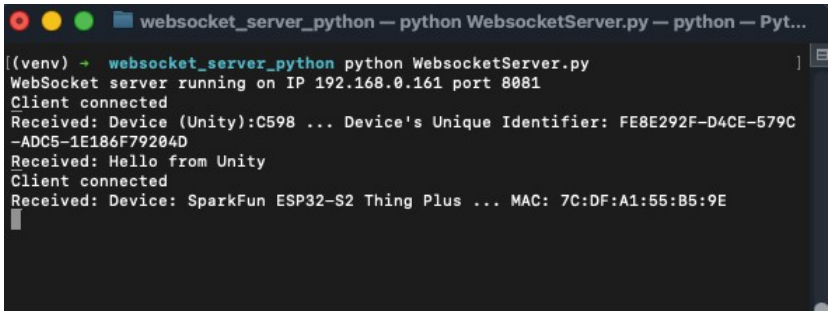


- If you correctly installed all the necessary libraries mentioned in the Required Preparation section, your code should have uploaded correctly. Open the Serial Monitor Window by clicking on the following Icon: (Or by going to **"Tools -> Serial Monitor"**



- You should see the following in your Serial Monitor, and if you check back to your Python Server, you should see that a new client has connected to it
  - I**mportant!:** If you are using the ESP32 S3 (Black one with 2 Micro USB Ports), make sure you connect your cable to the port that says **"UART"** next to it to be able to see the Serial prints

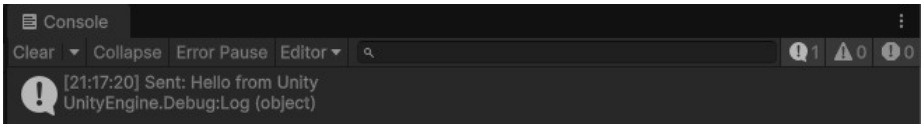


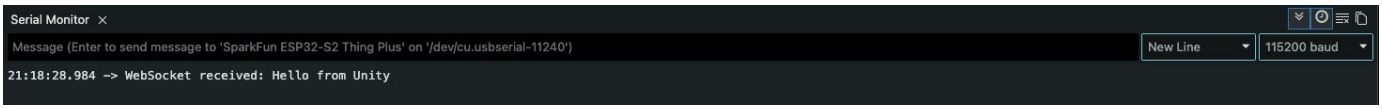


Task 5: Bi-directional communication between Unity and ESP32

We now have two clients connected to a single WebSocket server! Let's check if they can communicate with each other:

- In Unity, try once again clicking on the "Send test message" button
  - Checking Unity's console, you should see a Log saying "Sent: Hello From Unity"
  - Checking the Serial on Arduino IDE, you should see a message saying "WebSocket received: Hello From Unity"





- Now try selecting some of the other buttons in Unity, such as "Send LED ON Instruction". What changes have you observed in your ESP32? Is there anything different in the Arduino IDE Serial Monitor besides "WebSocket received: LED_INTENSITY:255"?
  - Take a few minutes to investigate the code from the "**03_Client_Example_Basic.ino**" file to try to understand what is happening

- Finally, grab your ESP32, and click on the Built-in Button that says "0", "BOOT" or "B" (different models in the lab will say different things)
  - In the Arduino's Serial Monitor, you should see "Button Up" or "Button Down" when pressing the button
  - In Unity, you should see "Received: button:0" or "Received: button:1", but also another Debug.Log saying "ESP32 Button Pressed"/"ESP32 Button Released"

Task 6: Understanding how Websocket Messages are handled in this example

Unity:

- In the previous Task, when pressing the built-in button in the ESP32 and sending a Websocket message to Unity, not only does Unity print in the console "Received:button:0"/"Received:button:1", but we also see a "ESP32 Button Pressed"/"ESP32 Button Released" log
- The reason for this is that we instruct Unity to do a "Debug.Log" when it detects that it's receiving a message that contains "button" and that message, depending on the value following "button", we ask it to print a specific log
- Our "IncomingMessageParser()" method is responsible for reacting to all messages received from other clients and triggering events in Unity

```
public void IncomingMessageParser(String msg)

{

string valueParsed = msg.Substring( msg.IndexOf(":") + 1);



if(msg.Contains("button")) {

if(valueParsed == "1")

{

//do something if button pressed

Debug.Log("ESP32 Button Pressed");

}

if(valueParsed == "0")

{

//do something if button released

Debug.Log("ESP32 Button Released");

}
```

```
 }


}
```

⚠️ **Note:** In this method, we have built a rudimentary system that first checks for a TYPE of message (i.e. button) and for a VALUE (i.e. 0 or 1) that are separated by a colon (i.e. ":"). We then we do something depending on the value received. In theory, all Websocket messages from clients should be structured in a "TYPE:VALUE" in our system from now on (e.g. "LED:100")

- You can build on top of this to later fit your needs, for example:

```
if(msg.Contains("Potentiometer") {
 int myInt = int.Parse(valueParsed); //converts string to int
//Do something with myInt
//Example:
RunMyCustomMethod(myInt);
}
```

- Similarly, we can create custom code that includes the "websocket.SendText()" method to call later and trigger conditions in the ESP32:

```
public async void SendLedIntensity()
{
if (websocket != null && websocket.State == WebSocketState.Open)
{
await websocket.SendText("LED_INTENSITY:" + ledIntensity);
Debug.Log("Sent: LED_INTENSITY:" + ledIntensity);
}
else
{
Debug.LogWarning("WebSocket not connected");
}
}
```

ESP32:

- In the ESP32 side of things, similar to Unity, we have a function responsible for handling the messages received by other clients called "handleMessage()":
- Similarly to Unity, this function is also structured in a way to recognize messages in a "TYPE:VALUE" way (e.g. LED_INTENSITY:255)

```
void handleMessage(const String& message) { // This function is set up to receive and parse messages in the form of "TYPE:VALUE" (e.g. Le
int sep = message.indexOf(':');
if (sep == -1) return;
```

```
 String type = message.substring(0, sep);
 int value = message.substring(sep + 1).toInt();

 if (type.equalsIgnoreCase("LED_INTENSITY")) { //If this ESP receives a Websocket message of type "LED_INTENSITY", then set the built in
 value = constrain(value, 0, 255);
 ledSet(value);
 Serial.printf("LED intensity → %d\n", value);
 }

 if(type.equalsIgnoreCase("CUSTOM WEBSOCKET MESSAGE")) {
 //Do something
 }

}
```

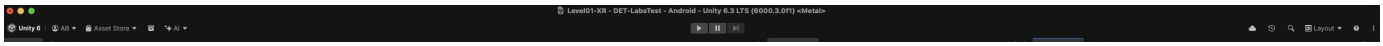
- As for sending messages, in the Arduino IDE we have a function similar to Unity, called "sendTXT()"
- Here, we can to include all our logic inside our loop function (e.g. when a button is pressed, when a potentiometer value is changed, etc) and when a condition is met, we want to trigger that Websocket message to be sent

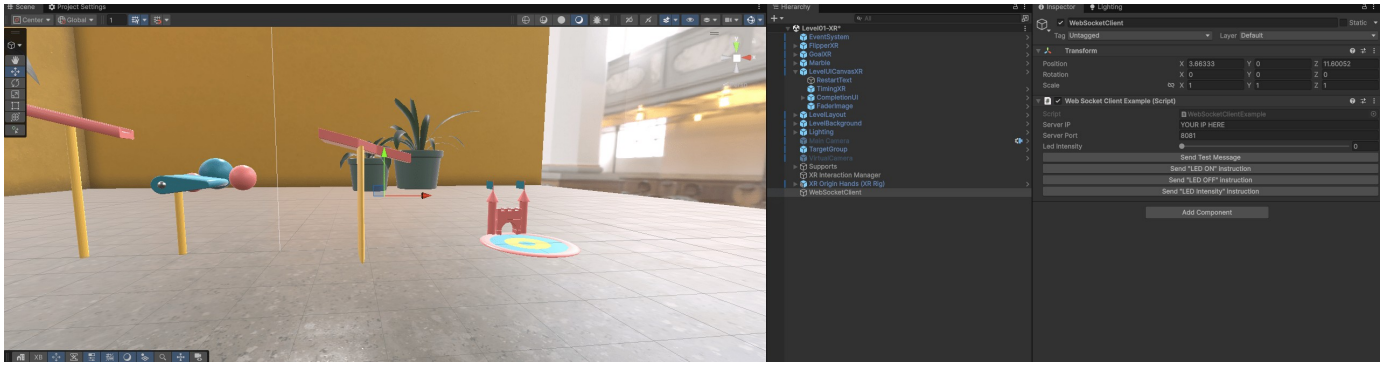
```
void loop() {
 webSocket.loop();

 bool currentButtonState = digitalRead(BUILTIN_BUTTON_PIN);

 if (lastButtonState == HIGH && currentButtonState == LOW) { //Sends Websocket Message when you push down the built in button
 webSocket.sendTXT("button:1");
 Serial.println("Button Down");
 }

 if (lastButtonState == LOW && currentButtonState == HIGH) { //Sends Websocket Message when you stop pushing the built in button
 webSocket.sendTXT("button:0");
 Serial.println("Button Up");
 }
 lastButtonState = currentButtonState;

}
```

Task 7: Modifying our previous Lab's sample to work with Websockets

Let's now make it possible to use WebSockets to control our Flipper from the previous Lab's Scene

- Open "Leve1-XR" scene located inside the "XR-Introduction/Lab-Exercises/Scenes" folder
- Right click on the hierarchy and select "Create Empty" to create a new GameObject and name is "WebSocketClient"
- Select it and then click on the "Add Component" button and search for the "Web Socket Client Example" script and add it to our Gameobject
- Edit the "Server IP" variable to match the IP in your Python server





The flipper is coded in a way that it is looking for an input from the New Input System from Unity, so we will need to modify it a bit in order for it to work with our WebSocket System:

- Open the "InteractivePuzzlePieceXR.cs" script. This is the class that "FlipperXR" inherits from and where most of the logic behind it is present.
- Create a new "public bool esp32Controlled" variable
- On line 25, replace the following code:

  if(interactActionReference.action.IsPressed() && m_IsControllable)
- With:

  if(interactActionReference.action.IsPressed() || esp32Controlled && m_IsControlable) //With this we can check for either a Unity Input or a Websocket input

Now open the "WebSocketClientExample" script and

- Create a new public variable "flipper" of the type "FlipperXR"
- Inside "IncomingMessageParser", replace the following code:

```
public void IncomingMessageParser(String msg)

{

string valueParsed = msg.Substring( msg.IndexOf(":") + 1);



if(msg.Contains("button")) { //If the first part of the websocket message contains "button"

if(valueParsed == "1")

{

//do something if button pressed

Debug.Log("ESP32 Button Pressed");

}

if(valueParsed == "0")

{

//do something if button released

Debug.Log("ESP32 Button Released");

}



}



}
```

- With:

```
public void IncomingMessageParser(String msg)

{

string valueParsed = msg.Substring( msg.IndexOf(":") + 1);
```

```
        if(msg.Contains("button")) { //If the first part of the websocket message contains "button"
         if(valueParsed == "1")
        {
        flipper.esp32Controlled = true;
        Debug.Log("Flipper Activated via WebSocket");
        }
        if(valueParsed == "0")
        {
        flipper.esp32Controlled = false;
        Debug.Log("Flipper Deactivated via WebSocket");
        }

        }

        }
```

⚠️ **Note:** Don't forget to drag and drop in the inspector the "FlipperXR" reference to our newly created variable!

- Run the game with both Unity and ESP clients connected to the Python server and press the built-in button on the ESP32. On some models, the button might be called "0" while on others it might be called "BOOT"/"B"

🎉 The flipper now moves! But we are missing sound!

- Add some extra lines to our code in "WebSocketClientExample":

```
if(msg.Contains("button")) { //If the first part of the websocket message contains "button"
if(valueParsed == "1")
{
flipper.esp32Controlled = true;
flipper.puzzleAudioSource.PlayOneShot(flipper.activateSound); //Triggers activation sound
Debug.Log("Flipper Activated via WebSocket");
}
if(valueParsed == "0")
{
flipper.esp32Controlled = false;
flipper.puzzleAudioSource.PlayOneShot(flipper.deactivateSound); //Triggers deactivation sound
Debug.Log("Flipper Deactivated via WebSocket");
}
```

⚠️ **Note:** The reason why we have to do this, is because the "InteractivePuzzlePieceXR" triggers the sound on the "Update()" and is using a couple of methods ("WasPressedThisFrame()" and "WasReleasedThisFrame()") to check for input. Since we do not have a way to check the exact frame our boolean value changed, we must instead replicate it using the above shown way.

Next steps (Lab Assignment)...

- Continue editing the "Level1-XR" scene using more interactions from the ESP32. You might need to collect some extra components such as sensors and actuators to finish these tasks
  - Add a way to also restart the game with the press of a button (hint: Check the "SceneCompletion" script in Unity)
  - Add a way to change the force or the angle that the flipper uses to hit the ball (hint: Check the "Hinge Joint" component in the flipper Gameobject and look at "Force" and "Limits". You might need to do some extra editing in the "FlipperXR" code to finish this task
  - Make the built-in LED (or add another one using a breadboard and jumper wires) turn on when you successfully complete the game. Don't forget to also make sure it is turned off if you restart the game!

Next steps (Home Assignment)...

- Convert your IDKI Unity minigame (exchange students will get a template instead) into a VR version of it that can also receive input or send commands to/from an ESP32

Last modified: Thursday, 5 February 2026, 09:43

◄ Required preparation Lab 5

Jump to...

List of Tangible Lab Resources ►